Marevlo Research / AI Foundations • Module 07 – Production • Topic 1

Kingman's Wait & the Load-Shedding Threshold, Worked Out by Hand

Why queueing latency explodes near saturation — and the exact utilisation at which to start shedding

What this document does. It takes one concrete production target — the *Nexus v7* fraud-detection assistant, one region serving up to 1100 req/s with a 250 ms time-to-first-token (TTFT) SLO — and derives, from Kingman's M/G/1 approximation, *why* latency grows nonlinearly as the box fills and *exactly* which utilisation ρ^* you must cap admission at to keep the SLO. Every number is produced by a small reference script (Appendix A), so the arithmetic is exact and reproducible. By the end you can compute, on paper, the gateway rate-limit value (∼919 req/s) that protects the SLO.

The one idea. A queue's wait time is not linear in load. The factor $\rho/(1-\rho)$ has a pole at $\rho = 1$: as utilisation approaches capacity, wait races to infinity. Load shedding is the act of refusing traffic *before* you reach the steep part of that curve.

Concepts covered, in order
the M/G/1 model and what ρ, CV_a, CV_s, $E[S]$ mean; Kingman's wait formula; why $\rho/(1-\rho)$ makes latency nonlinear; the danger point $\rho = 0.9$; solving for the shedding threshold ρ^* two ways (closed form and bisection); translating ρ^* into a gateway rate-limit; and why you shed before the knee, not after.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

Contents

1 The system under study

Production LLM serving is, underneath the GPUs, a queueing problem: requests arrive, wait in line for a slot, get served, and leave. We fix one concrete instance so every quantity is a number you can check.

Quantity	Symbol	Value here
Service capacity	μ	1100 req/s
Mean service time	$E[S]$	60 ms
Arrival variability (Poisson)	CV_a	1.0
Service-time variability (LLM)	CV_s	0.5
TTFT SLO	—	250 ms
Utilisation (the dial we turn)	ρ	λ/μ

Here λ is the arrival rate (req/s) and $\rho = \lambda/\mu$ is *utilisation*: the fraction of capacity in use. CV is the coefficient of variation (std. dev. ÷ mean) of a distribution; a perfectly regular stream has CV = 0, a Poisson stream has CV = 1. LLM service times are *less* variable than exponential ($\mathrm{CV}_s \approx 0.5$) because a token budget bounds how long any one request runs.

Intuition

A queue is like a single-lane toll booth. If cars arrive at 40% of the rate the booth can clear them, there's almost never a line. At 90%, a tiny clump of arrivals creates a backup that takes a long time to drain — because the booth is barely keeping up even on average. The closer arrival rate creeps to service rate, the more catastrophic each random burst becomes. That "closeness to capacity" is exactly ρ, and the wait blows up as $\rho \to 1$.

2 Step 1 — Kingman's wait approximation

For a general single-server queue (Poisson-ish arrivals, *general* service-time distribution — the "G" in M/G/1), Kingman gave a remarkably accurate closed form for the mean time a request spends *waiting* before service begins.

Kingman's M/G/1 expected wait

$$E[W] \;\approx\; \underbrace{\frac{\rho}{1-\rho}}_{\text{utilisation}} \times \underbrace{\frac{\mathrm{CV}_a^2 + \mathrm{CV}_s^2}{2}}_{\text{variability}} \times \underbrace{E[S]}_{\text{service}}.$$

The formula is a product of three independent knobs. The **utilisation** term $\rho/(1-\rho)$ is the one that explodes. The **variability** term $(\mathrm{CV}_a^2 + \mathrm{CV}_s^2)/2$ is a constant for a given workload — here

$$\frac{\mathrm{CV}_a^2 + \mathrm{CV}_s^2}{2} = \frac{1.0^2 + 0.5^2}{2} = \frac{1.25}{2} = 0.625.$$

The **service** term $E[S] = 60\,\text{ms}$ just sets the time scale. The *time a user actually perceives* to first token is the wait plus one service:

$$\mathrm{TTFT}(\rho) \;=\; E[W](\rho) + E[S].$$

Mini-example — the variability factor in isolation

The $(\mathrm{CV}_a^2 + \mathrm{CV}_s^2)/2$ factor says "how bursty is the world." Two deterministic streams ($\mathrm{CV}_a = \mathrm{CV}_s = 0$) give a factor of 0: a perfectly metronomic queue *never* waits, at any $\rho < 1$. Pure Poisson arrivals with exponential service ($\mathrm{CV}_a = \mathrm{CV}_s = 1$) give a factor of 1 — that is the classic M/M/1 result $E[W] = \frac{\rho}{1-\rho}E[S]$. Our LLM sits at 0.625, below M/M/1, because the token cap tames service variability. Less burst, less wait, same ρ.

3 Step 2 — Why wait grows nonlinearly

Hold the variability and service terms fixed and sweep ρ. The entire shape of the latency curve comes from $\rho/(1-\rho)$:

ρ	$\frac{\rho}{1-\rho}$	$E[W]$ (ms)	TTFT (ms)
0.40	0.667	25.0	85.0
0.50	1.000	37.5	97.5
0.60	1.500	56.3	116.3
0.70	2.333	87.5	147.5
0.80	4.000	150.0	210.0
0.90	9.000	337.5	397.5

The doubling that more-than-quadruples the wait

Move ρ from 0.4 to 0.8 — a 2× increase in load. The utilisation factor goes

$$\frac{0.4}{1-0.4} = 0.667 \quad\longrightarrow\quad \frac{0.8}{1-0.8} = 4.000,$$

so $E[W]$ goes from 25.0 ms to 150.0 ms:

$$\frac{E[W](0.8)}{E[W](0.4)} = \frac{4.000}{0.667} = 6.0\times .$$

The textbook shorthand "doubling ρ *quadruples* the wait" is the right intuition — wait grows *super-linearly*, far faster than load — but the exact multiplier depends on *where* on the curve you double. Near the pole it is even more violent: doubling $0.45 \to 0.90$ multiplies $E[W]$ by $\frac{9.0}{0.818} \approx 11\times$. The lesson is the same either way: the cost of load is not linear, and it accelerates as you approach capacity.

The pole at $\rho = 1$

$E[W] \propto \dfrac{\rho}{1-\rho}$ has a vertical asymptote at $\rho = 1$. As utilisation approaches 100%, the denominator $1 - \rho \to 0$ and the wait $\to \infty$. There is no "running hot at 99% to save

money" — 99% utilisation means a $\rho/(1-\rho) = 99$ multiplier, roughly 11× the wait you already considered unacceptable at $\rho = 0.9$. Capacity planning lives entirely on the left of this pole.

4 Step 3 — The danger point: $\rho = 0.9$

Suppose autoscaling lags and the region drifts to 90% utilisation. Plug in:

TTFT at $\rho = 0.9$

$$E[W](0.9) = \frac{0.9}{1-0.9} \times 0.625 \times 60\,\text{ms} = 9.0 \times 0.625 \times 60\,\text{ms} = 337.5\,\text{ms},$$

$$\text{TTFT} = E[W] + E[S] = 337.5 + 60 = 397.5\,\text{ms}.$$

Against the 250 ms SLO this is a **violation by** 147.5 ms — the SLO is missed by more than half. And 90% is not even "overloaded" in the naive CPU-dashboard sense; the box still has 10% headroom on paper. The queue says otherwise.

Watch out

A CPU/GPU-utilisation dashboard reading 90% looks healthy — "plenty of headroom." But the user-visible TTFT at $\rho = 0.9$ is already 397.5 ms, a hard SLO breach. Utilisation is a lagging, linear-looking signal hiding a nonlinear latency reality. This is why Module 07 autoscales on *queue depth* (a direct proxy for $\rho/(1-\rho)$), not on CPU.

5 Step 4 — Solving for the shedding threshold ρ^*

We want the *largest* ρ that still meets the SLO. Define ρ^* by $\text{TTFT}(\rho^*) = \text{SLO}$. Above ρ^* the gateway must shed; below it, admit freely.

5.1 Closed form

Invert Kingman for ρ^*

First subtract the one unavoidable service time to get the *wait budget*:

$$E[W]^* = \text{SLO} - E[S] = 250 - 60 = 190\,\text{ms}.$$

Set Kingman's wait equal to that budget and isolate the utilisation factor. Let $K = \dfrac{\rho^*}{1-\rho^*}$. Then

$$E[W]^* = K \cdot \frac{\text{CV}_a^2 + \text{CV}_s^2}{2} \cdot E[S] \implies K = \frac{E[W]^*}{0.625 \times 60\,\text{ms}} = \frac{190}{37.5} = 5.0667.$$

Finally invert $K = \rho/(1-\rho)$ via $\rho = K/(1+K)$:

$$\rho^* = \frac{K}{1+K} = \frac{5.0667}{6.0667} = 0.8352. \quad \checkmark$$

5.2 Bisection (independent check)

Mini-example — binary search, the way the artifact solves it

TTFT(ρ) is monotonically increasing in ρ, so a bisection on $[0, 1)$ converges to the unique root. Starting from the bracket and halving 60 times gives $\rho^* = 0.8352$, with TTFT(ρ^*) = 250.00 ms exactly on the SLO — agreeing with the closed form to machine precision ($|\Delta| \sim 10^{-16}$). Two independent methods, same answer: the threshold is real, not an algebra slip.

6 Step 5 — From ρ^* to a gateway rate-limit

A threshold on utilisation is not directly configurable; gateways throttle on *requests per second*. Convert:

Admission rate from the shedding threshold

$$\lambda^* \;=\; \rho^* \cdot \mu.$$

Plug in

$$\lambda^* = 0.8352 \times 1100\,\text{req/s} = 918.7\,\text{req/s} \;\approx\; \mathbf{919}\,\text{req/s}. \quad \checkmark$$

Configure the gateway's local rate-limit at ~919 req/s. Above it, shed with HTTP 503 and a clear `x-ratelimit-reason` header; shed in priority order — batch class first, interactive last. Note this caps admission at 919 even though the hardware *capacity* is 1100: the missing 181 req/s is the price of keeping TTFT under the SLO. You are deliberately leaving capacity on the table to stay off the steep part of the curve.

7 Step 6 — Why shed *before* the knee

What does each extra percent of utilisation past ρ^* actually cost?

ρ	TTFT (ms)	Verdict
0.835 ($= \rho^*$)	250.0	✓ on the SLO
0.920	491.3	breach (1.97× SLO)
0.950	772.5	breach (3.09× SLO)
0.980	1897.5	breach (7.59× SLO)

Just 8.5 points of utilisation above ρ^* (to 0.92) nearly *doubles* TTFT; 14.5 points triples it. The curve has a knee right where we placed the cap. Shedding a little traffic early — refusing the marginal request at 919 req/s — is overwhelmingly cheaper than letting the queue climb into the region where every admitted request, including the well-behaved ones already in flight, pays a multiplied wait.

The shedding discipline

Pick the SLO. Invert Kingman to get ρ^*. Multiply by capacity to get λ^*. Cap admission there and shed the rest. You are not throwing away "free" throughput — past ρ^* that throughput is poisoned, because admitting it breaks the SLO for *every* concurrent request, not just the marginal one.

8 The argument, at a glance

#	Step	Result
1	Kingman's wait	$E[W] = \frac{\rho}{1-\rho} \cdot \frac{\mathrm{CV}_a^2+\mathrm{CV}_s^2}{2} \cdot E[S]$
2	Nonlinearity	$\rho/(1-\rho)$ has a pole at $\rho = 1$; 0.4→0.8 is 6× wait
3	Danger point	$\rho = 0.9 \Rightarrow$ TTFT = 397.5 ms, breaches 250
4	Threshold	$\rho^* = K/(1+K) = 0.835$ (closed form = bisection)
5	Rate-limit	$\lambda^* = \rho^* \mu = 919$ req/s
6	Discipline	shed before the knee; past ρ^* throughput is poisoned

8.1 Equation cheat-sheet

Utilisation	$\rho = \lambda/\mu$
Kingman wait	$E[W] = \frac{\rho}{1-\rho} \cdot \frac{\mathrm{CV}_a^2+\mathrm{CV}_s^2}{2} \cdot E[S]$
Time to first token	$\mathrm{TTFT} = E[W] + E[S]$
Wait budget	$E[W]^* = \mathrm{SLO} - E[S]$
Utilisation factor	$K = E[W]^* / \left(\frac{\mathrm{CV}_a^2+\mathrm{CV}_s^2}{2} E[S] \right)$
Shedding threshold	$\rho^* = K/(1+K)$
Gateway rate-limit	$\lambda^* = \rho^* \, \mu$

Appendix A — Reference implementation

Every number above is produced by `m07_t01_kingman_load_shedding_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change μ, $E[S]$, the CVs, or the SLO to re-solve ρ^* for your own service.

```
"""
Reference implementation for: Module 07 - Topic 1
"Kingman's M/G/1 Wait & the Load-Shedding Threshold."

Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
so the arithmetic in the document is exact and self-consistent.

Scenario: the Nexus v7 fraud-detection assistant, one region, capacity
mu = 1100 req/s, mean service E[S] = 60 ms, Poisson arrivals (CV_a = 1.0),
LLM service-time variability CV_s = 0.5, TTFT SLO = 250 ms.

Convention: all times in seconds internally; printed in ms where noted.
"""

# ---------------------------------------------------------------------
# 0. The system under study.
# ---------------------------------------------------------------------
mu     = 1100.0        # service capacity, requests / second
ES     = 0.060         # mean service time E[S], seconds (60 ms)
CV_a   = 1.0           # coeff. of variation of arrivals (Poisson => 1)
CV_s   = 0.5           # coeff. of variation of LLM service times
SLO    = 0.250         # TTFT service-level objective, seconds (250 ms)

# The variability factor (CV_a^2 + CV_s^2)/2 is constant for this workload.
VAR = (CV_a**2 + CV_s**2) / 2.0
print("== 0. Workload constants ==")
print(f"capacity mu          = {mu:.0f} req/s")
```

```
print(f"mean service E[S]     = {ES*1e3:.0f} ms")
print(f"CV_a, CV_s            = {CV_a:.2f}, {CV_s:.2f}")
print(f"variability factor    = (CV_a^2 + CV_s^2)/2 = "
      f"({CV_a**2:.2f}+{CV_s**2:.2f})/2 = {VAR:.4f}")
print(f"TTFT SLO              = {SLO*1e3:.0f} ms")


# ---------------------------------------------------------------------
# 1. Kingman's M/G/1 wait approximation.
#    E[W] ~= (rho/(1-rho)) * (CV_a^2 + CV_s^2)/2 * E[S]
# ---------------------------------------------------------------------
def kingman_wait(rho):
    """Expected queueing wait E[W] in seconds for utilisation rho."""
    return (rho / (1.0 - rho)) * VAR * ES

def ttft(rho):
    """Time-to-first-token = queueing wait + one service time."""
    return kingman_wait(rho) + ES

print("\n== 1. Wait grows nonlinearly in rho ==")
print(f"{'rho':>6} {'rho/(1-rho)':>12} {'E[W] ms':>10} {'TTFT ms':>10}")
for rho in [0.4, 0.5, 0.6, 0.7, 0.8, 0.9]:
    print(f"{rho:>6.2f} {rho/(1-rho):>12.3f} "
          f"{kingman_wait(rho)*1e3:>10.2f} {ttft(rho)*1e3:>10.2f}")

# The "doubling rho quadruples wait" claim, exactly.
w04 = kingman_wait(0.40)
w08 = kingman_wait(0.80)
print(f"\nE[W](0.40) = {w04*1e3:.3f} ms")
print(f"E[W](0.80) = {w08*1e3:.3f} ms")
print(f"ratio E[W](0.8)/E[W](0.4) = {w08/w04:.3f}x  "
      f"(rho doubled -> wait ~{w08/w04:.0f}x)")

# ---------------------------------------------------------------------
# 2. The SLO-violating operating point (rho = 0.9).
# ---------------------------------------------------------------------
print("\n== 2. The danger zone: rho = 0.90 ==")
rho = 0.90
print(f"E[W](0.90) = (0.9/0.1) x {VAR:.4f} x {ES*1e3:.0f} ms "
      f"= {kingman_wait(rho)*1e3:.1f} ms")
print(f"TTFT       = E[W] + E[S] = {kingman_wait(rho)*1e3:.1f} + {ES*1e3:.0f} "
      f"= {ttft(rho)*1e3:.1f} ms")
print(f"SLO = {SLO*1e3:.0f} ms -> "
      f"{'PASS' if ttft(rho) <= SLO else 'VIOLATION'} "
      f"(over by {(ttft(rho)-SLO)*1e3:.1f} ms)")

# ---------------------------------------------------------------------
# 3a. Closed-form shedding threshold rho* (solve TTFT(rho*) = SLO).
#     E[W] = SLO - E[S];  let K = E[W]/(VAR*E[S]) = rho*/(1-rho*)
#     => rho* = K / (1 + K)
# ---------------------------------------------------------------------
print("\n== 3a. Closed-form rho* such that TTFT = SLO ==")
W_budget = SLO - ES                     # wait budget after subtracting service
K        = W_budget / (VAR * ES)        # = rho*/(1-rho*)
rho_star_cf = K / (1.0 + K)
print(f"wait budget  E[W]* = SLO - E[S] = {SLO*1e3:.0f} - {ES*1e3:.0f} "
      f"= {W_budget*1e3:.0f} ms")
print(f"K = E[W]*/(VAR*E[S]) = {W_budget*1e3:.0f}/({VAR:.4f}x{ES*1e3:.0f}) "
      f"= {K:.4f}")
print(f"rho* = K/(1+K) = {K:.4f}/{1+K:.4f} = {rho_star_cf:.4f}")

# ---------------------------------------------------------------------
# 3b. Binary search for rho* (matches the artifact's solver).
```

```
# ---------------------------------------------------------------------
def solve_rho_star(slo, lo=1e-6, hi=1-1e-9, iters=60):
    for _ in range(iters):
        mid = 0.5 * (lo + hi)
        if ttft(mid) <= slo:
            lo = mid
        else:
            hi = mid
    return lo

rho_star = solve_rho_star(SLO)
print("\n== 3b. Binary-search rho* (independent check) ==")
print(f"rho* (bisection) = {rho_star:.4f}")
print(f"TTFT(rho*)       = {ttft(rho_star)*1e3:.2f} ms  (== SLO {SLO*1e3:.0f})")
print(f"agreement vs closed form: |diff| = {abs(rho_star-rho_star_cf):.2e}")

# ---------------------------------------------------------------------
# 4. Translate rho* into a gateway rate-limit (req/s to admit).
# ---------------------------------------------------------------------
lam_star = rho_star * mu
print("\n== 4. Gateway rate-limit value ==")
print(f"lambda* = rho* x mu = {rho_star:.4f} x {mu:.0f} = {lam_star:.1f} req/s")
print(f"-> configure gateway local-rate-limit at ~{round(lam_star):d} req/s")
print(f"   (shed with HTTP 503 above this; batch class first, interactive last)")

# ---------------------------------------------------------------------
# 5. What shedding buys you: wait at the admit point vs unshed 0.95.
# ---------------------------------------------------------------------
print("\n== 5. Why shed BEFORE the knee ==")
for rho in [rho_star, 0.92, 0.95, 0.98]:
    print(f"  rho={rho:5.3f} -> TTFT={ttft(rho)*1e3:8.1f} ms "
          f"({'OK' if ttft(rho)<=SLO else 'BREACH'})")
print("Past rho*, every extra 1% utilisation costs disproportionately more wait;")
print("the gateway caps admission so the queue never enters the steep region.")
```

— END OF DOCUMENT —